

---

## Security Audit – Wormhole Native Token Transfers (NTT)

|               |                 |
|---------------|-----------------|
| Lead Auditor: | Benno Fünfstück |
|---------------|-----------------|

|                 |             |
|-----------------|-------------|
| Second Auditor: | Simon Klier |
|-----------------|-------------|

|                      |                 |
|----------------------|-----------------|
| Administrative Lead: | Thomas Lambertz |
|----------------------|-----------------|

April 12<sup>th</sup> 2024

The logo consists of the letters 'Nd' in a bold, black, sans-serif font, centered within a black square border. The background of the page features abstract geometric shapes in shades of yellow, orange, and grey.

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                      | <b>4</b>  |
|          | Engagement Details . . . . .                             | 4         |
|          | Findings Summary . . . . .                               | 4         |
|          | General Assessment . . . . .                             | 4         |
| <b>2</b> | <b>Project Overview</b>                                  | <b>5</b>  |
|          | Functionality . . . . .                                  | 5         |
|          | Transceiver . . . . .                                    | 5         |
|          | NTTManager . . . . .                                     | 5         |
|          | Programs . . . . .                                       | 6         |
|          | NTT Program . . . . .                                    | 6         |
|          | NTT Quoter Program . . . . .                             | 6         |
|          | Wormhole Governance Program . . . . .                    | 6         |
|          | On-Chain Data and Accounts . . . . .                     | 7         |
|          | NTT Program . . . . .                                    | 7         |
|          | NTT Quoter Program . . . . .                             | 8         |
|          | Wormhole Governance Program . . . . .                    | 8         |
|          | Instructions . . . . .                                   | 8         |
|          | NTT Program . . . . .                                    | 8         |
|          | NTT Quoter Program . . . . .                             | 9         |
|          | Wormhole Governance Program . . . . .                    | 10        |
|          | Upgrade Authority . . . . .                              | 10        |
|          | Admin Authority . . . . .                                | 10        |
|          | Security Features . . . . .                              | 10        |
| <b>3</b> | <b>Scope</b>                                             | <b>10</b> |
| <b>4</b> | <b>Findings</b>                                          | <b>12</b> |
|          | [ND-NTT01-I1] Safety Relies on Overflow-Checks . . . . . | 13        |
|          | [ND-NTT01-I2] Anchor Lint Errors . . . . .               | 14        |
|          | <b>Appendices</b>                                        |           |
| <b>A</b> | <b>About Neodyme</b>                                     | <b>16</b> |
| <b>B</b> | <b>Methodology</b>                                       | <b>17</b> |

**C Vulnerability Severity Rating**

**18**

# 1 | Introduction

## Engagement Details

Wormhole engaged Neodyme to do a detailed security analysis of the Solana programs of their Native Token Transfers (NTT) product.

Two senior security researchers from Neodyme, Benno Fünfstück and Simon Klier, conducted independent full audits of the contract between the 11th of March and the 29th of March.

Both auditors have a long track record of finding critical and other vulnerabilities in Solana programs as well as in Solana's core code itself. This expertise also extends to several other bridging implementations on Solana.

## Findings Summary

The number of findings identified through the audit, grouped by severity, can be seen below.



We are pleased to confirm that the engagement resulted in no findings above informational severity.

We publish the details of these issues in this report. In the following sections, we also give an overview of the NTT's architecture.

## General Assessment

Both the administrative and the developer teams of the NTT were very professional and responsive during the audit. The code quality was very good. It is a testament to the clear and overall robust design of the contract that only informational issues were identified.

## 2 | Project Overview

## Functionality

Wormhole's Native Token Transfers (NTT) is a framework for creating tokens that are transferrable across chains. Instead of traditional bridges like Wormhole's Portal, no liquidity pools are needed as the bridged tokens are minted directly/burned directly. The framework is also modular, allowing the user to integrate their own token and metadata protocol as well as the relay network.

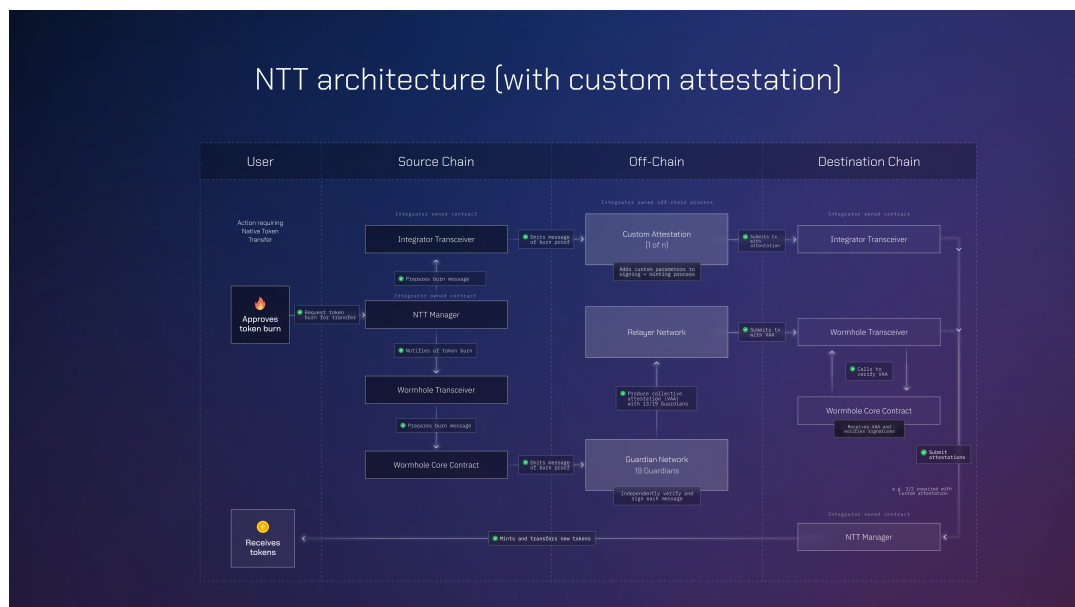
The two core components of NTTs are the **Transceiver** and the **NTTManager**.

## Transceiver

Tranceivers are responsible for sending the NTT transfers to the NTT manager on the target chain. Wormhole can be used as a transceiver but custom attestation protocols can also be realized by implementing the transceiver interface.

## NTTManager

The NTTManager handles the tokens and the transceivers. It is responsible for receiving token in- and outflows through the receivers and rate-limiting.



**Figure 1:** NTT architecture

## **Programs**

The scope contains three example NTT programs, a token, quoter and governance implementation.

### **NTT Program**

An example implementation of a NTTManager and a Wormhole traneiver. It can either run in “locking” mode where the token supply remains on a single chain and tokens have to be “locked” to be able to be transferred to another blockchain, or in “burning” mode where the token supply is distributed over multiple chains. The program manages the incoming/outgoing messages, rate-limiting/queuing and implements the traneiver interface for Wormhole.

This program is the main target of the audit.

### **NTT Quoter Program**

The NTT quoter is an example implementation of a relayer and handles fee collection.

### **Wormhole Governance Program**

The governance program can execute privileged instructions on behalf of the guardian set.

## On-Chain Data and Accounts

We give a brief overview of what data the three contracts store on-chain and what accounts/PDAs they use.

### NTT Program

The NTT Program has a main *Config* account which is the central account storing the state of and configuration of the NTT program:

- owner of the *Config*
- config of the token:
  - mint
  - token/token22 program
  - custodian
  - burning/locking mode
- chain id of the *Config*
- enabled transceivers
- pause authority

There are also various PDAs:

- **RegisteredTransceiver**
  - pubkey and ID of an transceiver
- **NTTManagerPeer**
  - address of a NTTManager on another chain
- **InboxRateLimit**
  - rate limit for incoming funds from a specific chain
- **OutboxRateLimit**
  - rate limit for outgoing funds (global, not chain specific)
- **InboxItem**
  - incoming message
  - contains amount, target and rate limit state
- **OutboxItem**

- outgoing message
- contains amount, timestamp and target information (chain ID, NTTManager, address)
- **TransceiverPeer**
  - address of a peer on another chain

### NTT Quoter Program

The *Instance* is the main account holding the state of the the Quoter. It stores:

- owner pubkey
- assistant pubkey (the assistant has almost the same power as the owner)
- fee recipient pubkey
- sol price in usd

PDA:

- **RegisteredChain**
  - contains fee information for a specific target chain
- **RelayRequest**
  - marker that the fee for a message has been paid

### Wormhole Governance Program

No state except for PDAs that act as replay protection.

## Instructions

We briefly summarize the instructions of the programs here for completeness.

### NTT Program

| Instruction  | Category       | Summary                                        |
|--------------|----------------|------------------------------------------------|
| Initialize   | Deployer only  | Initialize <i>Config</i> and outbox rate limit |
| TransferBurn | Permissionless | Transfer tokens in “burning” mode              |



| Instruction          | Category           | Summary                                                              |
|----------------------|--------------------|----------------------------------------------------------------------|
| TransferLock         | Permissionless     | Transfer tokens in “locking” mode                                    |
| Redeem               | Permissionless     | Put message into inbox                                               |
| ReleaseInboundMint   | Permissionless     | Remove deposit from inbox and mint tokens                            |
| ReleaseInboundUnlock | Permissionless     | Remove deposit from inbox and unlock tokens                          |
| TransferOwnership    | Owner only         | Change pending owner and remove upgrade authority                    |
| ClaimOwnership       | Pending Owner only | Make pending owner upgrade authority and owner                       |
| SetPaused            | Owner only         | Set pause state                                                      |
| SetPeer              | Owner only         | Create <i>NTTManagerPeer</i> and corresponding <i>InboxRateLimit</i> |
| RegisterTransceiver  | Owner only         | Register and enable transceiver                                      |
| SetOutboundLimit     | Owner only         | Set outbound rate limit                                              |
| SetInboundLimit      | Owner only         | Set inbound rate limit                                               |
| SetTransceiverPeer   | Owner only         | Initialize <i>TransceiverPeer</i>                                    |
| ReceiveMessage       | Permissionless     | Receive message from peer                                            |
| ReleaseOutbound      | Permissionless     | Post message from outbox to wormhole                                 |
| BroadcastId          | Permissionless     | Post NTTManager address and token mint address                       |
| BroadcastPeer        | Permissionless     | Post chain id and the corresponding peer                             |

### NTT Quoter Program

| Instruction   | Category             | Summary                           |
|---------------|----------------------|-----------------------------------|
| RegisterChain | Owner only           | Initialize <i>RegisteredChain</i> |
| CloseRelay    | Owner/Assistant only | Close <i>RelayRequest</i>         |
| RequestRelay  | Permissionless       | Pay fee for relay                 |

| Instruction       | Category             | Summary                                                |
|-------------------|----------------------|--------------------------------------------------------|
| UpdateSolPrice    | Owner/Assistant only | Update sol price                                       |
| UpdateChainPrices | Owner/Assistant only | Update prices of <i>RegisteredChain</i>                |
| UpdateChainParams | Owner/Assistant only | Update base fee and max gas for <i>RegisteredChain</i> |

### Wormhole Governance Program

| Instruction | Category       | Summary                         |
|-------------|----------------|---------------------------------|
| Governance  | Permissionless | Execute governance instructions |

## Upgrade Authority

The upgrade authority is automatically set to the owner of the *Config*.

### Admin Authority

The *Config* owner can register new transceivers and peers. But as the owner is also the upgrade authority of the program, it has full control over the funds. Compromising the owner would lead to loss of funds.

## Security Features

There is a global rate limit on inflows and rate limits per chain for inflows.

## 3 | Scope

The following project was in scope for this audit: Wormhole's Native Token Transfers Solana programs at revision 0d764bf7ffbbf81b4d51d1bfd5e51929e7fb0460, located at

- <https://github.com/wormhole-foundation/example-native-token-transfers>.

The repository is open source.

All on-chain components of the solana programs program were defined as in-scope. In particular, this includes the example NTT, NTT quoter and Wormhole governance program found at `solana/programs {Rexx}` and the NTT messages library at `solana/modules/ntt-messages`.

All third-party libraries were deemed out-of-scope.

## 4 | Findings

All findings are classified according to according to Neodymes **Rating Classification**. In total, the audit revealed:

- **0** critical issues,
- **0** high-severity issues,
- **0** medium-severity issues,
- **0** low-severity issues, and
- **2** informational issues.

Below, we list all findings identified throughout the audit. We provide details on each in the section hereafter.

| Name                                         | Severity      |
|----------------------------------------------|---------------|
| ND-NTT01-I1 Safety Relies on Overflow-Checks | Informational |
| ND-NTT01-I2 Anchor Lint Errors               | Informational |

**[ND-NTT01-I1] Safety Relies on Overflow-Checks**

| Severity    | Impact                                                             | Affected Component | Status   |
|-------------|--------------------------------------------------------------------|--------------------|----------|
| <b>Info</b> | Contracts become insecure if compiled with wrong compiler settings | All programs       | Resolved |

Usually, rust does not perform overflow checks in release builds. This often causes serious security vulnerabilities. With NTT, overflows would be possible in the bitmap used for registering the transceivers and the trimming/untrimming code. To prevent this, the Wormhole team has also enabled overflow checks in release builds, which is best practice. While these fix the mentioned issues when building the contracts directly, it is not possible to unconditionally enable these checks when including the code as a library. Therefore, third-party users of the code would be affected if they don't enable overflow checks themselves.

**Resolution**

The Wormhole engineering team acknowledged this finding and published a fix in commits 76211b2ea5a3e85dfcf93a7fe1c96fbf5a2ccb5c and 2f4c01a6add62ebef2f05e4287a19bb51aa7d35.

The Neodyme auditors verified the fix.

**[ND-NTT01-I2] Anchor Lint Errors**

| Severity    | Impact                  | Affected Component | Status   |
|-------------|-------------------------|--------------------|----------|
| <b>Info</b> | Anchor lints aren't run | All programs       | Resolved |

Anchor v0.29 only runs linting on `anchor idl parse` (or if the IDL is generated via this method during build). The NTT example project however uses the `idl-build` feature which does not go through `anchor idl parse`. This means that lints were not checked during build.

The git version of anchor fixes this issue. When using that version, the following lint errors are found, but all of them are safe and do not cause security issues:

| location                                                                                     | message                                                            |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| example-native-token-transfers/src/instructions/admin.rs:92:4                                | Struct field “upgrade_lock” is unsafe, but is not documented.      |
| example-native-token-transfers/src/instructions/admin.rs:207:8                               | Struct field “transceiver” is unsafe, but is not documented.       |
| example-native-token-transfers/src/instructions/initialize.rs:61:8                           | Struct field “token_authority” is unsafe, but is not documented.   |
| example-native-token-transfers/src/instructions/release_inbound.rs:32:8                      | Struct field “token_authority” is unsafe, but is not documented.   |
| example-native-token-transfers/src/instructions/transfer.rs:112:8                            | Struct field “session_authority” is unsafe, but is not documented. |
| example-native-token-transfers/src/instructions/transfer.rs:208:8                            | Struct field “session_authority” is unsafe, but is not documented. |
| example-native-token-transfers/src/transceivers/wormhole/instructions/broadcast_id.rs:27:8   | Struct field “emitter” is unsafe, but is not documented.           |
| example-native-token-transfers/src/transceivers/wormhole/instructions/broadcast_peer.rs:31:8 | Struct field “emitter” is unsafe, but is not documented.           |
| wormhole-governance/src/instructions/governance.rs:55:8                                      | Struct field “program” is unsafe, but is not documented.           |

### **Resolution**

The Wormhole engineering team acknowledged this finding and is working on a fix in pull request #330 which has not been merged as of writing of this report (12th of April 2024).

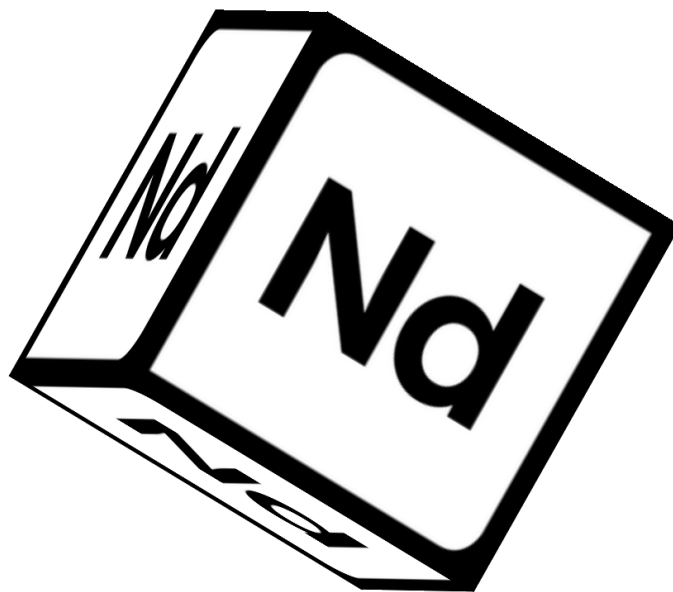
## A | About Neodyme

Security is difficult.

To understand and break complex things, you need a certain type of people. People who thrive in complexity, who love to play around with code, and who don't stop exploring until they fully understand every aspect of it. That's us.

Our team never outsources audits. Having found over 80 High or Critical bugs in Solana's core code itself, we believe that Neodyme hosts the most qualified auditors for Solana programs. We've also found and disclosed critical vulnerabilities in many of Solana's top projects and have responsibly disclosed issues that could have resulted in the theft of over \$10B in TVL on the Solana blockchain.

All of our team members have a background in competitive hacking. During such hacking competitions, called CTFs, we competed and collaborated while finding vulnerabilities, breaking encryption, reverse engineering complicated algorithms, and much more. Through the years, many of our team members have won national and international hacking competitions, and keep ranking highly among some of the hardest CTF events world-wide. In 2020, some of our members started experimenting with validators and became active members in the early Solana community. With the prospect of an interesting technical challenge and bug bounties, they quickly encouraged others from our CTF team to look for security issues in Solana. The result was so successful that after reporting several bugs, in 2021, the Solana Foundation contracted us for source code auditing. As a result, Neodyme was born.





## B | Methodology

Neodyme prides itself on not being a checklist auditor. We adapt our approach to each audit, investing considerable time into understanding the program upfront and exploring its expected behavior, edge cases, invariants, and ways in which the latter could be violated. We use our uniquely deep knowledge of Solana internals, and our years-long experience in auditing Solana programs to even find bugs that others miss. We often extend our audit to cover off-chain components, in order to see how users could be tricked or the contract affected by bugs in those components.

Nonetheless, we also have a list of common vulnerability classes, which we always exhaustively look for. We provide a sample of this list below.

- Rule out common classes of Solana contract vulnerabilities, such as:
  - Missing ownership checks
  - Missing signer checks
  - Signed invocation of unverified programs
  - Solana account confusions
  - Redeployment with cross-instance confusion
  - Missing freeze authority checks
  - Insufficient SPL account verification
  - Missing rent exemption assertion
  - Casting truncation
  - Arithmetic over- or underflows
  - Numerical precision errors
- Check for unsafe design which might lead to common vulnerabilities being introduced in the future
- Check for any other, as-of-yet unknown classes of vulnerabilities arising from the structure of the Solana blockchain
- Ensure that the contract logic correctly implements the project specifications
- Examine the code in detail for contract-specific low-level vulnerabilities
- Rule out denial of service attacks
- Rule out economic attacks
- Check for instructions that allow front-running or sandwiching attacks
- Check for rug pull mechanisms or hidden backdoors

## C | Vulnerability Severity Rating

Neodyme assesses the severity of findings using the following general vulnerability classification system. Note that bugs are considered on a case-by-case basis and that we may deviate from these guidelines in exceptional circumstances.

### **CRITICAL**

Vulnerabilities that will likely cause loss of funds. An attacker can trigger them with little or no preparation, or they are expected to happen accidentally. Effects are difficult to undo after they are detected.

### **HIGH**

Bugs that can be used to set up loss of funds in a more limited capacity, or to render the contract unusable.

### **MEDIUM**

Bugs that do not cause direct loss of funds but that may lead to other exploitable mechanisms, or that could be exploited to render the contract partially unusable.

### **LOW**

Bugs that do not have a significant immediate impact and could be fixed easily after detection.

### **INFO**

Bugs or inconsistencies that have little to no security impact.

**Neodyme AG**

Dirnismaning 55

Halle 13

85748 Garching

E-Mail: [contact@neodyme.io](mailto:contact@neodyme.io)

<https://neodyme.io>